PyTorchによるTwo-Towerモデル構築ベースライン

> **目的：** 800次元の行動ベクトルと100次元の商品ベクトルを、それぞれ独立したニューラルネットワーク（タワー）に通して**共通の128次元の潜在空間**に変換します。
>
> これにより、次元の異なるデータ同士でも数学的に正しくコサイン類似度を算出できるようになり、スコアが特定の値（0.15など）に極端に集中してしまう現象を解決できます。

1. ライブラリのインポートとダミーデータの準備

実際の運用では、ユーザーが「クリックした/購入した」商品ペアを正例（ラベル `1.0`）、「無視した/無関係」な商品ペアを負例（ラベル `-1.0`）としてデータセットを作成します。

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# --- 1. ダミーデータの生成 ---
num_samples = 1000
user_dim = 800  # A: 行動特徴
item_dim = 100  # B: 商品特徴

# ランダムな行動特徴と商品特徴を生成
X_user = torch.randn(num_samples, user_dim)
X_item = torch.randn(num_samples, item_dim)

# 正例(似ている=1.0)と負例(似ていない=-1.0)のラベルをランダム生成
# ※実データでは、購買履歴などのログから作成します
y = torch.randint(0, 2, (num_samples,)).float() * 2 - 1

dataset = TensorDataset(X_user, X_item, y)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

2. Two-Towerモデルの定義

それぞれのタワーで次元を変換した直後に、`nn.functional.normalize` を使ってベクトルを長さ1に正規化します。これにより、両ベクトルの内積を計算するだけで、自動的にコサイン類似度（-1.0〜1.0）が出力されます。

```
# --- 2. モデルの定義 ---
class TwoTowerModel(nn.Module):
    def __init__(self, user_dim=800, item_dim=100, embed_dim=128):
        super(TwoTowerModel, self).__init__()

        # 行動特徴用タワー (800次元 -> 256次元 -> 128次元)
        self.user_tower = nn.Sequential(
            nn.Linear(user_dim, 256),
            nn.ReLU(),
            nn.Linear(256, embed_dim)
        )

        # 商品特徴用タワー (100次元 -> 128次元 -> 128次元)
        self.item_tower = nn.Sequential(
            nn.Linear(item_dim, 128),
            nn.ReLU(),
            nn.Linear(128, embed_dim)
        )

    def forward(self, user_features, item_features):
        # 1. 各特徴を共通の潜在空間(128次元)に変換
        u_embed = self.user_tower(user_features)
        i_embed = self.item_tower(item_features)

        # 2. L2正規化 (ベクトルの長さを1にする)
        u_embed = nn.functional.normalize(u_embed, p=2, dim=1)
        i_embed = nn.functional.normalize(i_embed, p=2, dim=1)
```

```
        # 3. 内積を計算（正規化済みなので結果はコサイン類似度）
        cosine_sim = torch.sum(u_embed * i_embed, dim=1)

        return cosine_sim

model = TwoTowerModel()
```

3. 損失関数と最適化手法の設定

モデルが出力したコサイン類似度が、正解ラベル（`1.0` または `-1.0`）にどれくらい近いかを測るため、平均二乗誤差（MSELoss）を使います。

```
# --- 3. 損失関数と最適化手法 ---
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

4. 学習ループと推論のテスト

用意したデータをモデルに流し込み、誤差を計算してパラメータを更新する処理を繰り返します。

```
# --- 4. 学習ループ ---
epochs = 10

for epoch in range(epochs):
    total_loss = 0.0
    for batch_user, batch_item, batch_labels in dataloader:
        optimizer.zero_grad()

        # モデルにデータを入力し、コサイン類似度を予測
        predictions = model(batch_user, batch_item)

        # 損失(Loss)の計算とパラメータ更新
        loss = criterion(predictions, batch_labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")

print("学習完了！\n")

# --- 推論のテスト ---
test_user = torch.randn(1, 800)
test_item = torch.randn(1, 100)
model.eval()
with torch.no_grad():
    sim_score = model(test_user, test_item)
    print(f"テストデータのコサイン類似度: {sim_score.item():.4f}")
```